

Samruddhi Agros

– App Ideation Document

1. Overview

Samriddhi Agros is a hyperlocal quick-commerce platform delivering fresh farm produce directly from the farm to the customer's doorstep. The platform operates within a 15km radius of the farm, ensuring freshness, fast delivery, and manageable logistics at MVP stage.

The platform is inspired by quick-commerce leaders (Blinkit, Zepto, Swiggy Instamart, BigBasket) but tailored for perishable, farm-fresh produce rather than packaged goods.

2. Platform Structure

The platform is divided into 3 apps, each serving a distinct user role:

App	Purpose
Customer App	Browse, order, track, and pay for produce
Delivery Partner App	Accept orders, navigate, deliver, track earnings
Admin App	Manage products, pricing, offers, and platform-wide operations

3. Service Area

- . Platform operates with a single farm as the source for all produce
 - . Delivery restricted to a 15km radius from the farm
 - . Keeps delivery times short (critical for perishables)
 - . Keeps delivery partner fleet size manageable at launch
-

4. Customer App

4.1 Core Features

- . Clean, user-friendly interface (Swiggy/Zepto/Blinkit-style browsing)
- . Product listing with live pricing
- . Weight-based pricing – produce sold by kg/gram, not fixed units (e.g., ₹40/kg with 250g/500g/1kg/2kg selection)
- . Add to cart, edit quantities
- . Offers & discounts clearly visible
- . Real-time order status updates (placed → packed → out for delivery → delivered)
- . Live map integration to track delivery partner location during active delivery
- . Order history and easy reorder
- . Search, filter, and sort by category

4.2 Additional Recommended Features

- . Delivery slot selection (morning/evening) – important for produce freshness; decide whether the platform is instant-delivery (10–20 min) or slot-based, as this changes the entire ops model
 - . Push/SMS notifications at each order stage
 - . Ratings & reviews (product quality + delivery experience)
 - . "Report an issue" flow with photo upload for damaged/poor-quality produce, linked to refund/replacement
 - . Wishlist/favorites
 - . Clear cancellation policy (e.g., no cancellation once farm has picked/packed the order)
 - . In-app customer support (chat or helpline)
 - . Referral/loyalty program
-

5. Delivery Partner App

5.1 Core Features

- . Dashboard showing orders accepted/rejected
- . Daily/weekly/monthly earnings tracker
- . Map navigation: farm → customer doorstep
- . Real-time location sharing (active during delivery only, for privacy)

5.2 Onboarding & Verification

- . Mandatory KYC before onboarding:
 - Aadhaar/PAN verification (DigiLocker API – free, standard for Indian platforms)
 - Driving license verification
 - Vehicle details
 - Bank/UPI details for payouts
- . Manual or automated approval step before activation

5.3 Additional Recommended Features

- . Incentive structure: surge pay during peak hours, daily/weekly bonus targets
 - . Accept/reject order timer (standard quick-commerce UX pattern)
 - . In-app support for delivery issues
-

6. Admin App

6.1 Core Features

- . Platform-wide visibility into all running operations
- . Add/edit/remove products
- . Update pricing
- . Enable discounts:
 - Original price shown with **strikethrough**
 - Discounted price shown alongside
- . Manage offers and promotional campaigns

6.2 Additional Recommended Features

- . Role-based access control (e.g., support staff cannot edit prices; only authorized admins can)
- . Audit logs for every admin action (who changed what price, when)
- . Delivery partner approval/rejection dashboard
- . Order dispute/refund management panel
- . Sales and demand analytics (best-selling produce, peak order times)
- . Inventory/batch management dashboard:
 - View all batches with Best Before dates and current status (Active / Flagged / Extended / Perished)
 - Notification center for batches needing inspection
 - "Extend" and "Mark as Perished" action buttons per flagged batch
 - Perished Items log for wastage tracking

7. Pricing & Offers Logic

Rule	Detail
Free delivery	On purchases $\geq$ ₹100
Delivery charge	Applied if order value $<$ ₹100

Rule	Detail
Discount tier	5% off on purchases $\geq$ ₹500 (extendable to further tiers, e.g., 8% at ₹1000)
Discount display	Strikethrough original price + discounted price shown together

8. Payments

- . UPI (via Razorpay/PhonePe – supports UPI + COD, no fixed integration cost, small per-transaction fee only)
 - . Cash on Delivery (COD)
 - . Transaction status tracked per order (success/failed/pending)
-

9. Perishability & Inventory Lifecycle (Produce $\neq$ Packaged Goods)

Since there is no cold storage, produce has a limited sellable window. Every batch is tagged with a "Best Before" date, and inventory status updates through a mix of automatic system checks and manual admin decisions.

9.1 Inventory Status Flow

1. Active (In Stock) – Produce is live on the customer app and sellable, tracked with a Best Before date per batch.

2. **Best Before date reached (Automatic)** – The system automatically flags the batch and moves it out of the sellable listing. It does not delete the stock immediately.
3. **Admin Notification (Automatic)** – A message/alert is auto-sent to the admin:
"Batch X of Product Y has crossed its Best Before date – inspection required."
4. **Physical Inspection (Manual)** – The admin physically inspects the flagged produce and makes one of two decisions:
 - **Usable** → Admin clicks "Extend", marking the batch sellable for a defined grace period (max 1-2 days). It re-enters the Active listing, clearly tagged (e.g., "Sell today") until the extended window lapses.
 - **Not usable** → Admin clicks "Mark as Perished", and the batch is moved to a Perished Items log, removed from active stock, and quantity is deducted/deleted from sellable inventory.
5. **Extended window lapses without sale (Automatic)** – If a batch that was manually extended still isn't sold by the end of the grace period, the system automatically re-flags it for a final admin check (repeat of step 3-4), since indefinite auto-extension isn't allowed.

9.2 Manual vs. Automatic Update Summary

Action	Trigger Type
Stock reduces as orders are placed	Automatic
Batch flagged when Best Before date is reached	Automatic
Notification sent to admin on flagging	Automatic
Physical quality inspection	Manual (admin, in person)
Extend sellable window (max 1-2 days)	Manual (admin action)
Mark batch as Perished & remove from stock	Manual (admin action)
Perished quantity deducted from inventory records	Automatic (once admin confirms)
Admin manually adds/edits stock (new harvest, corrections)	Manual

9.3 Additional Notes

- . Quality-related refund/replacement workflow for customers, since spoilage/damage is more common than with packaged goods
- . Delivery slot system to preserve freshness if not doing instant delivery

- Perished Items log retained for record-keeping (helps track wastage trends over time, useful for admin analytics later)

10. Technical Approach (Minimal / Zero Cost MVP)

Layer	Suggested Tool	Cost
Customer + Delivery apps	Flutter / React Native (single codebase, two apps)	Free
Admin panel	React / Next.js web dashboard	Free
Backend	Node.js + Express, or Django	Free
Database	PostgreSQL (Supabase free tier) or MongoDB Atlas free tier	Free (within limits)
Caching	Redis via Upstash – reduces DB load during traffic spikes	Free tier
Hosting	Render / Railway / Fly.io	Free tier
Maps	Mapbox or Google Maps (\$200/mo free credit)	Free initially

Layer	Suggested Tool	Cost
Push notifications	Firebase Cloud Messaging	Free
Auth	Firebase Auth / Supabase Auth	Free tier
Payments	Razorpay / PhonePe	No fixed cost, transaction fee only
Traffic protection	Cloudflare (free tier)	Free
Order queueing	BullMQ + Redis to prevent write bottlenecks during spikes	Free

Note: Zero-cost is achievable for MVP and early testing, but at scale (heavy traffic, higher DB usage, higher map API calls), some paid infrastructure will likely become necessary. Load-test on free tiers before any major marketing push, and budget roughly \$20–50/month once free-tier limits are crossed.

11. Database Structure (Key Tables)

- **Users** – id, name, phone, email, password_hash, created_at
- **Addresses** – id, user_id, address_line, lat, long, is_default

- . **DeliveryPartners** – id, name, phone, KYC docs, vehicle info, verification_status, bank/UPI details, rating, is_active
- . **Admins** – id, name, role, permissions
- . **Farm** – id, name, location (single-farm setup; table kept for clean structure)
- . **Products** – id, name, category, unit_type (kg/piece), base_price, discount_price, discount_%, image_url, is_active
- . **Batches** – id, product_id, harvested_on, best_before_date, quantity, status (active / flagged / extended / perished), extended_until, flagged_at, inspected_by (admin_id), inspected_at
- . **PerishedItems** – id, batch_id, product_id, quantity, marked_perished_at, marked_by (admin_id), reason (optional note)
- . **Cart** – id, user_id, product_id, qty
- . **Orders** – id, customer_id, delivery_partner_id, total_amount, discount_applied, delivery_charge, payment_mode, payment_status, order_status, delivery_slot, timestamps
- . **OrderItems** – id, order_id, product_id, qty, price_at_purchase
- . **Offers** – id, type (flat/%), min_purchase_amount, discount_value, valid_from, valid_to

- . Transactions – id, order_id, upi_ref, amount, status
 - . Ratings – id, order_id, product_rating, delivery_partner_rating, comment
 - . Notifications – id, user_id, message, type, is_read
-

12. Security & Privacy

- . Passwords hashed with bcrypt (never stored in plain text)
- . HTTPS/TLS enforced across the platform
- . JWT-based auth with short-lived access tokens + refresh tokens; OTP login for added security
- . Role-based access control (RBAC) strictly enforced on the admin app
- . Rate limiting on all public-facing APIs (also helps prevent crashes under heavy/abusive traffic)
- . Server-side input validation/sanitization to prevent SQL injection and XSS
- . Delivery partner KYC via DigiLocker (Aadhaar/PAN) and driving license verification
- . Masked/anonymized customer phone numbers for delivery partner calls (or route through in-app chat)
- . Delivery partner location tracked only during active delivery, not persistently

- . DPDP Act 2023 (India) compliance: explicit consent for data collection, right to request data deletion, minimal data retention
 - . Secrets (API keys, DB credentials) stored server-side via environment variables only
 - . Audit logs for all admin actions (price/discount changes, product edits)
-

13. Pros & Cons Summary

Pros

- . Clear separation of concerns via 3 apps
- . Hyperlocal radius model suits perishable goods well
- . Proven pricing/discount mechanics from established quick-commerce players
- . Verification-first delivery partner onboarding builds trust and safety
- . MVP achievable at near-zero cost using free-tier tools

Cons / Risks

- . "Zero server crashes" and "zero cost" are in tension – true high-availability infrastructure has a cost once traffic scales
- . Perishable goods carry higher operational risk (spoilage, quality complaints) than packaged quick-commerce goods

- . Deciding between instant delivery vs. slot-based delivery significantly changes the technical and operational model – needs to be locked in early
 - . No cold storage means the inventory lifecycle (flag → inspect → extend/perish) depends on timely admin action; delayed inspection could lead to unnecessary wastage
-

14. Open Decisions to Finalize

1. Instant delivery (10–20 min) vs. scheduled delivery slots?
2. In-house delivery fleet vs. gig-based delivery partners?
3. Which payment gateway (Razorpay vs. PhonePe vs. others) based on fee structure?
4. Exact grace period for "Extend" (fixed 1 day, fixed 2 days, or admin-selectable between the two at time of extension)?